Chapitre 8

Systèmes distribués

Par Emmanuel NGUIMBUS

Contenu du cours

Introduction
I. Caractéristiques principales des systèmes distribués
II. Les 8 formes de transparence des systèmes distribués
III. Architecture classique d'un système distribué
IV. Architectures réelles des systèmes distribués
V. Mécanismes de communication des systèmes distribués
VI. Défis des systèmes distribués
VII. Modèles de cohérence : Garantir une vue cohérente des données réparties
VIII. Consensus dans les systèmes distribués
IX. Théorème CAP
X. Horloges logiques dans les systèmes distribués
XI. Sécurité dans les systèmes distribués
XII. Avantages et inconvénients des systèmes distribués
Conclusion

Introduction

Introduction

Définition

Encore appelé système reparti, un **système distribué** est un ensemble de machines **autonomes**, reliées par un réseau, qui collaborent pour exécuter une application comme si elle fonctionnait sur un **seul ordinateur**.

Caractéristiques essentielles :

- Chaque machine a son **propre processeur** et, souvent, sa **propre mémoire**.
- Elles communiquent via **messages**, réseau local ou Internet.
- Le système cherche à masquer la **complexité** : l'utilisateur perçoit un service unique.

Exemples:

- Applications bancaires distribuées
- Cloud computing
- Microservices
- Systèmes IoT
- Réseaux sociaux

Introduction

Pourquoi distribuer un système ?

Un système distribué permet de dépasser les limites d'une machine unique.
Les motivations principales sont **performance**, **fiabilité**, **scalabilité** et **partage**.

N	Raison	Description
1.	**Augmenter les performances**	• Plus de nœuds = plus de puissance de calcul. • Plusieurs machines exécutent des tâches **en parallèle**, réduisant le temps global. • Exemple : calcul scientifique sur clusters, moteur de recherche Google.
2.	**Tolérance aux pannes**	• La panne d'un composant n'entraîne pas la panne du système. • Si un serveur tombe, d'autres continuent. • La disponibilité reste élevée (**High Availability**). • Exemple : Clusters Web, Bases de données répliquées
3.	**Scalabilité horizontale**	• On adapte la taille du système à la demande. • Augmenter la capacité en ajoutant de nouveaux nœuds. • Fondamental dans le Cloud (AWS, Azure, GCP). • Exemple : Kubernetes autoscaling, Microservices

Introduction

Pourquoi distribuer un système ?

Un système distribué permet de dépasser les limites d'une machine unique.
Les motivations principales sont **performance**, **fiabilité**, **scalabilité** et **partage**.

N	Raison	Description
4.	**Partage des ressources**	• Tout le monde accède aux ressources, où qu'elles soient. • Plusieurs utilisateurs accèdent aux mêmes données depuis différents endroits. • Exemples: Systèmes de fichiers réseau (NFS, Ceph), Bases de données distribuées, Stockage cloud (S3 bucket, Google Cloud Storage)
5.	**Proximité avec l'utilisateur**	• Rapprocher le calcul de l'utilisateur améliore l'expérience. • Réduire la latence en plaçant les services au plus près (CDN, Edge computing). • Exemple : vidéo YouTube servie par le data center le plus proche.

I. Caractéristiques principales des systèmes distribués

I. Caractéristiques principales des systèmes distribués

Donner une vision synthétique des principales caractéristiques attendues d'un système distribué moderne.

N	Caractéristique	Description
1.	**Interopérabilité**	Capacité pour différents systèmes (OS, langages, architectures CPU) de communiquer et de fonctionner ensemble. • Support de standards : **HTTP/HTTPS, JSON, REST, gRPC(Google RPC), TCP/IP** • Compatibilité hétérogène : Linux ↔ Windows ↔ x86 • Importance pour les microservices et le cloud
2.	**Partage des ressources**	Les ressources (données, fichiers, services, calculs) sont accessibles depuis plusieurs nœuds. • Ex : NFS, services web, bases de données distribuées • Gestion des accès concurrentiels • Contrôle d'accès et cohérence
3.	**Scalabilité**	Capacité à augmenter la puissance du système au fur et à mesure de la demande. **Deux formes :** • **Verticale** (scale-up): augmenter la puissance d'une machine • **Horizontale** (scale-out): ajouter de nouvelles machines (le plus utilisé) **Exemples :** Kubernetes autoscaling, serveurs Web avec Load Balancer

I. Caractéristiques principales des systèmes distribués

Donner une vision synthétique des principales caractéristiques attendues d'un système distribué moderne.

N	Caractéristique	Description
4.	**Tolérance aux pannes**	Le système continue de fonctionner même si un ou plusieurs nœuds tombent en panne. • Réplication des données • Détection et remplacement automatique des nœuds • Requêtes reroutées vers d'autres serveurs **Exemples :** HAProxy, réplication MySQL, Cassandra
5.	**Performance / latence**	La performance dépend des échanges réseau entre nœuds. • Minimiser les temps de communication • Optimiser le placement des données (localité) • Monitoring de latence inter-nœuds Ex : CDN(Content Delivery Network), microservices → forte sensibilité à la latence
6.	**Transparence**	Masquer la complexité du système distribué à l'utilisateur final. **Plusieurs formes de transparence :** • d'accès (utilisation uniforme) • de localisation (où est la donnée ? → pas visible) • de migration • de réplication • d'échec • de performance • de scalabilité • Ex : Kubernetes cache la localisation réelle des pods.

II. Les 8 formes de transparence des systèmes distribués

II. Les 8 formes de transparence des systèmes distribués

N	Forme de transparence	Description
1.	**Accès**	Même l'API indépendamment de la localisation. Ex : lire un fichier local ou NFS.
2.	**Localisation**	L'utilisateur ignore où se trouve la ressource. Ex : DNS, Kubernetes Services.
3.	**Migration**	Déplacement de ressources sans impact. Ex : migration de VM.
4.	**Réplication**	Plusieurs copies, mais vue unifiée. Ex : systèmes de réplication MySQL.
5.	**Échec**	Les défaillances sont masquées autant que possible. Ex : retry automatique d'un microservice.
6.	**Persistance**	Les données persistent indépendamment du traitement.
7.	**Performance**	Ajustement dynamique des performances sans changer le programme.
8.	**Scalabilité**	Ajout de nœuds sans reconfiguration majeure. Ex : clusters Kubernetes.

III. Architecture classique d’un système distribué

III. Architecture classique d'un système distribué

Un système distribué comporte généralement :

N	Niveau	Description
1.	**Niveau matériel**	• Nœuds (serveurs, VMs, conteneurs) • Réseau (LAN, WAN, Internet)
2.	**Niveau système / middleware**	• OS (Linux), hyperviseurs • Protocoles : TCP, HTTP, RPC • Middleware : gRPC, Kafka, Service Mesh
3.	**Niveau applicatif**	• Microservices • API • Traitements distribués

IV. Architectures réelles des systèmes distribués

IV. Architectures réelles des systèmes distribués

N	Type d'architecture	Description
1.	**Client–Serveur**	• Serveur central + clients. • Simple mais limité en scalabilité.
2.	**3-tiers**	• Client → Serveur d'application → Base de données. • Standard en entreprise.
3.	**Microservices**	• Application découpée en services indépendants. • Communication via REST/gRPC/Kafka • Idéal pour la scalabilité + la maintenance.
4.	**P2P**	• Chaque nœud joue le rôle client + serveur. • Exemple : Torrent, Blockchain.
5.	**Event-driven**	• Communication via messages / événements. • Ex: Kafka, RabbitMQ, MQTT

V. Mécanismes de communication des systèmes distribués

Dans un système distribué, les nœuds doivent échanger des informations.

Comment la communication entre les nœuds se fait-elle?

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

1. Appels à procédure distance (RPC : Remote Procedure Call)

Permet à un programme d'appeler une fonction située sur une autre machine comme si elle était locale.

Ex: gRPC (Google Remote Procedure Call)

- Développé par Google
- Très performant
- Idéal pour les microservices rapides et les communications internes

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

2. Communication via fichiers

Les nœuds partagent des fichiers sur un stockage distribué.

Technologies exploitant la communication via les fichiers

- **NFS** (Network File System)
- **CephFS**
- **HDFS** (Hadoop Distributed File System)

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

3. Middleware distribué / Coordination

Des services intermédiaires assurent la cohérence, la distribution, le routage, la découverte, etc.

Technologies utilisant la coordination

- **ZooKeeper** (coordination Hadoop, Kafka)
- **Etcd** (stockage clé-valeur dans Kubernetes)
- **Consul** (service discovery)

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

4. Service Mesh

Couche réseau intelligente qui gère automatiquement la communication inter-microservices :

Fonctionnalités offertes:

•**Communication entre microservices**

•**Sécurité** : mTLS automatique

•Load balancing intelligent

•Retries automatiques et timeouts

•Les développeurs se concentrent sur l'application et **le mesh gère le réseau**.

•**Ex: Kubernetes** : le service mesh gère la communication entre pods.

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

5. Messagerie asynchrone (Message Brokers)

Un **Message Broker** est un logiciel intermédiaire qui stocke, distribue ou diffuse les messages entre services distribués.
Composant qui reçoit, stocke et distribue des messages
Découple producteurs et consommateurs

Rôles :
- **Découpler** les systèmes émetteur (*publish)* et récepteur (*subscribe)* de messages
- Assurer la **fiabilité** (replay, persistency)
- Répartir la charge
- Supporter les architectures **Event-Driven**

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

5. Messagerie asynchrone (Message Brokers)

Exemples de Message Brokers:

a. Kafka (Apache Kafka)

•Message broker **haut débit** (jusqu'à millions d'événements/s)

•Stockage persistant → données conservées plusieurs jours

•Utilisé pour : logs, streams, analytics, pipelines temps réel

•Très utilisé dans : LinkedIn, Uber, Netflix

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

5. Messagerie asynchrone (Message Brokers)

Exemples de Message Brokers:

b. RabbitMQ

•MQ= “Message Queue”

•Orienté **files d’attente**

•Messages envoyés à une **queue**, consommés par un worker

•Idéal pour : tâches longues, distribution de charge, microservices

V. Mécanismes de communication des systèmes distribués

On distingue plusieurs grandes catégories de mécanismes :

5. Messagerie asynchrone (Message Brokers)

Exemples de Message Brokers:

c. MQTT (Message Queuing Telemetry Transport)

•Protocole **ultra-léger** pour IoT

•Utilise un modèle **publish/subscribe**

•Idéal pour : capteurs, objets connectés, domotique

6. REST — Representational State Transfer

Un style d'API **HTTP** largement utilisé.

Caractéristiques :

•Méthodes HTTP : GET, POST, PUT, DELETE

•Utilise JSON

•Très simple, universel

Utilisé partout : APIs web, microservices simples

VI. Défis des systèmes distribués

VI. Défis des systèmes distribués

Problèmes classiques des systèmes distribués:

N	Problème
1.	**Latence réseau non négligeable**
2.	**Bande passante limitée : congestion**
3.	**Horloges non synchronisées**
4.	**Pannes partielles** plus difficiles diagnostiquer qu'une panne totale
5.	**Données incohérentes** en cas de réplication
6.	**Sécurité distribuée plus complexe**
7.	**Débogage difficile** à cause du multi-nœuds

VII. Modèles de cohérence : Garantir une vue cohérente des données réparties

VII. Modèles de cohérence : Garantir une vue cohérente des données réparties

Pourquoi la cohérence est un problème ?

Dans un système distribué, **les données sont copiées sur plusieurs nœuds** pour la disponibilité et la performance.

Mais ces copies peuvent **ne pas être synchronisées au même instant**, ce qui crée des incohérences.

Les modèles de cohérence définissent ce qu'un client est autorisé à voir lorsqu'il lit une donnée.

VII. Modèles de cohérence : Garantir une vue cohérente des données réparties

1. Cohérence Forte (Strong Consistency)

Définition

- Après chaque écriture, **toutes les lectures retournent immédiatement la dernière valeur**.
- **Le système se comporte comme une machine unique non distribuée.**

Ex : Google Spanner (SGBD distribué orienté Cloud)

- **Avantage :** comportement intuitif
- **Inconvénient :** latence réseau élevée, faible scalabilité

2. Cohérence Séquentielle (Sequential Consistency)

Définition :
Toutes les opérations apparaissent comme exécutées dans **un ordre global**, mais **pas nécessairement l'ordre temps réel**.

Ex :
Si un programme écrit A puis B, tous les nœuds voient A avant B.
Mais l'opération peut être retardée dans le temps.

- **Avantage :** modèle fort mais moins coûteux que Strong
- **Inconvénient :** toujours difficile à implémenter dans les très gros clusters

3. Cohérence Causale (Causal Consistency)

Définition
Si une opération **B dépend causalement** d'une opération **A**,
alors **tous les nœuds doivent voir A avant B**.

Exemple d'un réseau social :
•Alice poste *"Bonjour !"*
•Bob répond *"Salut Alice !"*
Tous les utilisateurs doivent voir *"Bonjour !"* avant *"Salut Alice !"*
(même si les serveurs sont différents)

- **Avantage :** cohérence logique
- **Inconvénient :** ne garantit pas un ordre global entre opérations indépendantes

4. Cohérence Éventuelle (Eventual Consistency)

Définition :
Si aucune nouvelle écriture n'a lieu, **toutes les copies convergent** vers la même valeur, mais **on ne garantit pas ce que voit un client pendant la convergence**.

Exemples :
•Amazon DynamoDB
•S3
•DNS
•Cassandra (selon configuration)

- **Avantage :** super scalable, haute disponibilité
- **Inconvénient :** lectures temporaires incohérentes

VII. Modèles de cohérence : Garantir une vue cohérente des données réparties

Modèle	Lecture immédiate	Ordre global	Scalabilité	Exemple
Forte	✓	✓	✗	Google Spanner
Séquentielle	✓	⚠ (ordre logique)	✗	Simulations scientifiques
Causale	⚠	✗	✓	Réseaux sociaux
Éventuelle	✗	✗	✓✓	DynamoDB, S3

- **Plus un modèle est cohérent, moins il est scalable.**
- **Les systèmes modernes (Cloud, NoSQL) utilisent souvent la cohérence éventuelle** pour maximiser disponibilité et performance.

VIII. Consensus dans les systèmes distribués

VIII. Consensus dans les systèmes distribués

Objectif du consensus

Le consensus sert de protocole **d'accord** dans un système distribué.

Permettre à **plusieurs nœuds** d'un système distribué de **se mettre d'accord** sur une valeur commune, même en cas de :

- Pannes partielles
- Retards réseau
- Partitions réseau
- Messages perdus

Le consensus est **la base** :

- Des bases de données distribuées
- Du maintien d'un état cohérent dans un cluster.

VIII. Consensus dans les systèmes distribués

Pourquoi le consensus est difficile ?

Dans un système distribué :

- les messages peuvent arriver en retard
- les messages peuvent disparaître
- un nœud peut tomber
- un nœud peut être lent
- le réseau peut se partitionner

Impossible de savoir si un nœud est mort… ou juste down.

Les nœuds doivent donc **négocier** un état commun.

VIII. Consensus dans les systèmes distribués

Principaux protocoles de consensus distribué

1. Paxos — Le protocole historique

•Théoriquement robuste mais **complexe**
•Ancien standard en consensus
•Utilisé dans certains systèmes Google historiques
•Basé sur :

- Proposers
- Acceptors
- Learners

- Trop difficile à implémenter.

VIII. Consensus dans les systèmes distribués

Principaux protocoles de consensus distribué

2. Raft — Le consensus distribué moderne

Raft a été conçu pour :
•être **simple à expliquer**
•être **simple à implémenter**
•fournir les mêmes garanties que Paxos.

Raft repose sur 3 mécanismes :

- **Élection de leader:** Un nœud est choisi comme leader et coordonne les mises à jour.
- **Réplication du log:** Les entrées sont ajoutées dans le même ordre sur tous les nœuds.
- **Sécurité (Safety):** Aucune décision contradictoire n'est possible.

Aujourd'hui, Raft est **le standard des systèmes distribués modernes**.

VIII. Consensus dans les systèmes distribués

Principaux protocoles de consensus distribué

3. Byzantine Fault Tolerant (BFT)

- Résiste aux **nœuds malveillants**
- Utilisé dans la **Blockchain**
- Très complexe

VIII. Consensus dans les systèmes distribués

Principaux protocoles de consensus distribué

Exemple concret : Kubernetes et Raft

Dans Kubernetes :
•**etcd** stocke l'état du cluster
•etcd utilise **Raft**
•Le leader Raft est responsable :
- de valider les écritures
- de synchroniser les logs
- de maintenir la cohérence du cluster

- Si le leader tombe :
  Un **nouveau leader** est élu automatiquement et le cluster continue.

IX. Théorème CAP

IX. Théorème CAP

Définition

Le théorème CAP affirme qu'un système distribué **ne peut garantir simultanément** les trois propriétés suivantes :

C — Cohérence (Consistency)

Tous les nœuds voient la même donnée au même moment.
Ex : une écriture doit être immédiatement visible partout.

A — Disponibilité (Availability)

Chaque requête reçoit une réponse, même en cas de panne de certains nœuds.

P — Tolérance aux partitions (Partition Tolerance)

Le système continue de fonctionner malgré des coupures réseau ou une perte de communication entre nœuds.

En cas d'une partition réseau, un système doit choisir : la Cohérence ou la Disponibilité.

IX. Théorème CAP

Scénario CAP

1. CP — Cohérence + Tolérance aux partitions

Le système préfère la cohérence, quitte à sacrifier temporairement la disponibilité.

Comportement
Le système préfère **bloquer une requête** plutôt que risquer de retourner une donnée incohérente.

Ex:
MongoDB mode distribué : si le nœud primaire tombe en panne et qu'il n'y a pas de consensus entre autres nœuds opérationne
Il y a arrêt des écritures dans la BD.

IX. Théorème CAP

Scénario CAP

2. AP — Disponibilité + Tolérance aux partitions

Le système répond toujours aux requêtes même en cas de partition, mais les données peuvent être temporairement incohérentes.

Comportement
Chaque réplique accepte les écritures: les données convergent plus tard.

Ex: Cassandra: haute disponibilité, réplication multi-datacenters.

IX. Théorème CAP

Scénario CAP

3. CA — Cohérence + Disponibilité

Impossible dans un système réellement distribué avec risque de partitions.

On parle de CA s'il n'y a *pas de partition possible*.

Ex:

- Bases de données locales (PostgreSQL, MySQL sur une seule machine)
- Systèmes centralisés (sans réplication)

IX. Théorème CAP

Synthèse CAP

- CAP n'est pas un *choix*, mais une **contrainte fondamentale** des systèmes distribués.
- Les systèmes distribués tels que Kubernetes, Kafka, NoSQL positionnent leurs **choix CAP** selon leurs besoins.
- Le Cloud utilise souvent AP pour absorber les pannes massives.

Modèle	Choix	Sacrifice
CP	Cohérence	Disponibilité
AP	Disponibilité	Cohérence forte
CA	Cohérence + disponibilité	Pas tolérant aux partitions

X. Horloges logiques dans les systèmes distribués

X. Horloges logiques dans les systèmes distribués

Pourquoi les horloges logiques sont utiles dans un système distribué ?

Dans un système distribué, les machines :

- N'ont **pas la même heure**,
- Peuvent être **désynchronisées**,
- Exécutent des actions **en parallèle**,
- Echangent des messages avec **latence variable**.

Impossible d'utiliser l'heure système pour savoir **quel événement s'est produit avant un autre**.

Problème à résoudre : l'ordre des événements

Exemples concrets :

- Deux utilisateurs modifient **le même document** depuis deux serveurs différents.
- Deux microservices appliquent des mises à jour **dans un ordre différent**.
- Un message arrive **hors ordre** à cause du réseau.

Pour éviter les conflits, il faut **reconstruire l'ordre logique des événements**, même si les horloges physiques sont fausses.

Qu’est-ce qu’une horloge logique?

Une horloge logique attribue à chaque événement un **numéro croissant** qui permet de déterminer :

- **A précède B ?**
- **A et B sont indépendants ?**
- **B dépend de A ?**

Deux solutions :

1. Horloge de Lamport
2. Horloges vectorielles

Les type d'horloges logiques

1. Horloge de Lamport

Principe :
•On incrémente un compteur à chaque événement local.
•Lors d'un envoi/réception de message, on synchronise les compteurs.

Utilité :
- Permet de savoir **quel événement arrive avant un autre**.
- Utilisée pour : logs distribués, microservices, bases simples.

- Permet d'établir une relation :
  A se produit avant B si timestamp(A) < timestamp(B).

- **Limite**

Ne permet **pas** de détecter la concurrence.

Les types d'horloges logiques

2. Horloges vectorielles

Principe :
•Chaque nœud maintient un **vecteur** (une liste de compteurs).
•Permet de détecter les **événements concurrents**.

Utilité :
→ Résolution de conflits : systèmes de fichiers, NoSQL, synchronisation.
→ Exemples : DynamoDB, Cassandra, Git (concept similaire).

Permet de détecter la concurrence :
•Si V(A) < V(B) alors **A vient avant B**
•Si aucun n'est inférieur à l'autre → **événements concurrents** (indépendants)

✓ **Avantage clé**
Permet de savoir **quand deux mises à jour conflictuelles doivent être résolues (mais ne les résout pas)**.

X. Horloges logiques dans les systèmes distribués

Exemple:

Deux serveurs mettent à jour le même profil utilisateur.

Serveur A :
update(name="ISJ") → horloge 5

Serveur B :
update(city="Yaoundé") → horloge 3

Un troisième serveur reçoit :
1.name="Moussa", horloge=5
2.city="Yaoundé", horloge=3

- Le troisième serveur sait que **5 > 3**, donc l'événement "Moussa" est plus **récent**.
- Aucun risque d'écraser une modification avec une plus ancienne.

Technologies distribuées utilisant les horloges logiques

- **Git**
- **Dropbox**
- **Google Drive**

Limites des horloges dans les systèmes distribués

Les horloges logiques permettent de déterminer :
•Si deux événements sont **ordonnés causalement**
•Ou s'ils sont **concurrents**

✓ les horloges logiques servent à détecter les conflits

✗ les horloges logiques **ne résolvent pas les conflits**

✗ les horloges logiques ne permettent pas de fusionner proprement deux modifications concurrentes

Google Docs n'utilise pas les horloges logiques !

X. Horloges logiques dans les systèmes distribués

Caractéristique	Horloge Lamport	Horloge Vectorielle
Détecte l'ordre causal (Si l'événement A "cause" l'événement B, alors tous les nœuds doivent voir A avant B.) A → B → C → D	✓ Oui	✓ Oui
Détecte la concurrence	✗ Non	✓ Oui
Complexité	Très faible	Moyenne
Taille de l'horloge	1 entier	N entiers
Utilisation	systèmes simples	NoSQL, systèmes distribués modernes

XI. Sécurité dans les systèmes distribués

XI. Sécurité dans les systèmes distribués

Problèmes critiques :

- Authentification inter-nœuds
- Gestion des identités distribuées
- Propagation des sessions
- MITM(man In the Middle) sur RPC
- Besoin de chiffrement systématique

Solutions :

- OAuth2
- JWT
- Mutual TLS
- Zero Trust architecture
- Vault (gestion de secrets)

XII. Avantages et inconvénients des systèmes distribués

XII. Avantages et inconvénients des systèmes distribués

Avantages	Inconvénients
• Disponibilité accrue • Scalabilité horizontale • Déploiement flexible • Maintenance simplifiée (microservices) • Utilisation optimale des ressources • Adapté aux architectures Cloud / DevOps	• Complexité du développement • Débogage difficile • Gestion des conflits et incohérences • Latence réseau • Pannes partielles difficiles • Cohérence coûteuse (CAP) • Sécurité distribuée

Conclusion

Conclusion

Les systèmes distribués sont omniprésents :

- Cloud computing
- IA
- Big Data
- Applications web critiques
- Serveurs d’entreprise
- IoT

Conclusion

Les systèmes distribués permettent :

✔ Haute disponibilité
✔ Performance
✔ Tolérance aux pannes
✔ Scalabilité

Mais introduisent :

⚠️ Complexité
⚠️ Latence
⚠️ Cohérence difficile
⚠️ Sécurité complexe

FIN